Optimal Leaf Ordering against Trait Distances: a First-Principles Analysis

This document analyses the supplied dynamic program `olo_phylo` as a research candidate. Nothing about the published Bar-Joseph recurrence, the implementation's state, or the claimed exactness is taken as given. Every claim is classified as a theorem, a counterexample, or an empirical observation.

Intended-use assumptions, stated up front and used only where named. The implementation is designed for a *rooted, fully bifurcating* ape `phylo` object and a *symmetric* pairwise dissimilarity produced by `stats::dist`. Those two hypotheses are *not* part of the combinatorial problem until they are invoked. They will be shown to be necessary for exactness of *this* recurrence.

1 Precise formalization of the optimization problem

1.1 Input

A pair (T, d):

1. **Tree.** A rooted tree $T = (V, E)$ with leaf set $L(T) = \{1, \dots, n\}$, $n \geq 2$, in which every internal node has **exactly two children**. Branch lengths are not part of the input. Write $L(v)$ for the leaves in the subtree rooted at v, $\lambda(v) = |L(v)|$, and $\mathrm{lc}(v), \mathrm{rc}(v)$ for the two children of an internal node v. The unique root is ρ. There are exactly $n - 1$ internal nodes.

2. **Dissimilarity.** A real $n \times n$ matrix d. No metric axioms, no non-negativity, and no symmetry are required by the *problem*. (The *algorithm* will later impose symmetry.)

The supplied driver builds d from a trait matrix $X \in \mathbb{R}^{n \times p}$ by optional column-standardization and Euclidean `dist`. That is one data-generating process for d, not a restriction on the feasible set.

1.2 Decision variables

At each internal node v a binary **flip** $\phi(v) \in \{0, 1\}$ chooses which child is drawn on the left:

$$\pi_v \;=\; \begin{cases} \pi_{\mathrm{lc}(v)} \,\|\, \pi_{\mathrm{rc}(v)} & \text{if } \phi(v) = 0, \\ \pi_{\mathrm{rc}(v)} \,\|\, \pi_{\mathrm{lc}(v)} & \text{if } \phi(v) = 1, \end{cases}$$

with $\pi_\ell = (\ell)$ for a leaf ℓ. The global layout is $\pi = \pi_\rho$. Equivalently, ϕ ranges over $\{0, 1\}^{n-1}$, so there are **exactly** 2^{n-1} feasible layouts. They are pairwise distinct as permutations: flipping v swaps two nonempty disjoint blocks.

No other moves are permitted. In particular the topology of T is immutable: leaves of any subtree must occupy a contiguous block of π (Lemma 1.1).

1.3 Feasibility constraints

A permutation π of $L(T)$ is **T-consistent** if it arises from some ϕ, iff for every node v the set $L(v)$ is a consecutive interval of π. The family $\{L(v)\}_v$ is laminar, so the consecutive-block constraints are laminar.

Lemma 1.1 (contiguous blocks)**.** *For every T-consistent π and every node v, the image $\pi^{-1}(L(v))$ is an interval of $\{1, \ldots, n\}$.*

Proof. Induction on $\lambda(v)$. A leaf is an interval of length 1. At an internal node the two child blocks are consecutive by the inductive hypothesis and are concatenated, hence their union is consecutive. □

1.4 Objective

$$J(\pi) \;=\; \sum_{i=1}^{n-1} d\big(\pi(i),\, \pi(i+1)\big).$$

There is **no wrap-around term** $d(\pi(n), \pi(1))$. The problem is a *linear* (path) layout, not a circular phylogram layout. The objective does not depend on branch lengths, on the depth of nodes, or on any internal-node coordinates.

1.5 Output, and what "globally optimal" means

A feasible $\pi^\star$ (equivalently a flip vector $\phi^\star$) together with the scalar $J(\pi^\star)$, such that

$$J(\pi^\star) \;=\; \min\big\{J(\pi) : \pi \text{ is } T\text{-consistent}\big\}.$$

Ties may be broken arbitrarily. The implementation returns one such order as tip-index permutation and the scalar cost, then (separately) asks `ape::rotateConstr` to realize the order geometrically. The last step is not part of the combinatorial problem.

1.6 Structural properties of the problem

- **Finite exact combinatorial optimization** over a set of size 2^{n-1}.
- **Not** unconstrained Hamiltonian path / TSP: the laminar consecutive-block constraints are what make a polynomial algorithm possible. Unconstrained min-cost Hamiltonian path on d is NP-hard and has a larger feasible set.
- **Reversal.** If $d = d^\top$, then $J(\mathrm{rev}(\pi)) = J(\pi)$ for every π, because the undirected adjacent pairs are identical. If d is not symmetric this identity fails, and $\mathrm{rev}(\pi)$ need not even have a simple relation to $J(\pi)$. Reversal of a T-consistent order is always T-consistent (flip every internal node).
- **No greedy-choice property.** A locally cheapest arrangement of a proper subtree need not extend to a global optimum (Theorem 5.2, Counterexample 5.3).
- **Optimal substructure holds only after the interface is recorded.** The interface of a subtree with the rest of the layout is the ordered pair of endpoint leaves of its contiguous block (Section 5).

- **Laminar / hierarchical.** The subsets that ever occupy a contiguous block are exactly the sets $L(v)$. This is the difference between this DP and Held–Karp TSP (which ranges over all 2^n subsets).

1.7 Implicit assumptions of the implementation (not of the problem)

The code, as written, additionally assumes:

Assumption	Where it enters	If violate
T is rooted	unique node of in-degree 0	unrooted cating ar
T is strictly binary	`length(kids) != 2` is fatal	polytomi the feasil *changes*
d is symmetric	`L_full[colnames, rownames] <- t(LT)` identifies $M(u,w)$ with $M(w,u)$	inexact; is not th (Counter
d is indexed by tip numbers `"1"`..`"n"`, not by `tip.label`	`make_trait_dist` relabels after matching	calling `o` matrix si
finite numeric d	`which.min` / `== min_cost`	`scale()` `NaN` and argmin
$n \geq 2$	`rtree` contract	a one-lea

Most mathematically faithful reading of ambiguities.

- The recurrence is the *minimization* dual of Bar-Joseph et al. (2001), who *maximize* a similarity S. With d in place of $-S$ (or $c - S$) the argmin/argmax coincide. The code is a min-cost program, not a max-similarity program.
- "Left"/"right" children in ape's edge list are an arbitrary planar embedding. The DP searches all embeddings obtainable by child-swaps; it does not depend on the input embedding except as a starting point.
- The root table stores only the orientation "original-left child on the left". The reverse orientation is implicitly attributed the same cost. This is valid whenever the subtree optimum is **reversal-invariant**. Symmetry $d = d^\top$ is a simple sufficient condition and is the condition satisfied by the supplied `make_trait_dist` pipeline. It is not a logically necessary condition for every individual instance: some asymmetric matrices can happen to induce reversal-invariant optimum tables on a particular tree.

2 The existing recurrence, from first principles

2.1 States

For every node v the implementation stores a rectangular table whose rows are $L(\mathrm{lc}(v))$ and whose columns are $L(\mathrm{rc}(v))$. Write

$$M_v(u, w) \qquad u \in L(\mathrm{lc}(v)),\ w \in L(\mathrm{rc}(v))$$

for the stored entry. A dense "full" matrix F_v over $L(v) \times L(v)$ is then materialized by

$$F_v(u, w) \;=\; \begin{cases} 0 & \text{if } \lambda(v) = 1, \\ M_v(u, w) & \text{if } (u, w) \text{ is a stored pair,} \\ M_v(w, u) & \text{if } (w, u) \text{ is a stored pair (the transpose fill),} \\ +\infty & \text{otherwise.} \end{cases}$$

Intended meaning of $M_v(u, w)$. The minimum of J restricted to the contiguous block of $L(v)$, over T-consistent arrangements of that block that start at u and end at w, *with* $\mathrm{lc}(v)$ *placed to the left of* $\mathrm{rc}(v)$.

Intended meaning of $F_v(u, w)$. The same quantity with *either* orientation of v's two children allowed. The identification $F_v(w, u) = F_v(u, w)$ is the reversal shortcut.

2.2 State variables, listed

Symbol	Domain	Role
v	nodes of T	which laminar block is being arranged
u	a leaf of the left child	left endpoint of the block
w	a leaf of the right child	right endpoint of the block
$M_v(u, w) \in \mathbb{R} \cup \{+\infty\}$	scalar	best internal path cost for that interface
$B_v(u, w)$, $C_v(u, w)$	leaves	argmin seam witnesses, for traceback

No other history (flip vector, internal order, ancestor context) is stored.

2.3 Transition

Let $L = \mathrm{lc}(v)$, $R = \mathrm{rc}(v)$. The code computes, for every $u \in L(L)$ and $w \in L(R)$,

$$M_v(u, w) \;=\; \min_{\substack{b \in L(L) \\ c \in L(R)}} \Big(F_L(u, b) \;+\; F_R(c, w) \;+\; d(b, c) \Big), \tag{1}$$

skipping pairs (u, b) for which $F_L(u, b) = +\infty$ (equivalently: skipping pairs that are not a valid endpoint pair of L). The inner minimum over c is vectorized; the outer minimum over b is an R loop. Witnesses $(b^\star, c^\star)$ are stored as `ptrB`, `ptrC`.

2.4 Base case

A leaf ℓ: $L(\ell) = \{\ell\}$, $F_\ell(\ell, \ell) = 0$, and there is no table M. The empty path has cost 0.

2.5 Reconstruction

At the root, cost $= \min M_\rho$, and a minimizing stored pair $(u^\star, w^\star)$ is selected (the first, in the event of ties). Traceback:

- if the requested (s, t) is a stored (left-start, right-end) pair, concatenate `get_order(L, s, B(s,t))` with `get_order(R, C(s,t), t)`;
- if (s, t) is the reverse of a stored pair, return the reverse of `get_order(v, t, s)`;
- a leaf returns itself.

The reconstructed sequence is claimed to be a T-consistent order of cost equal to the reported minimum.

2.6 Does (R) consider every feasible solution?

Every T-consistent order of $L(v)$ with left child on the left and endpoints (u, w) has a unique seam (b, c): b is the last leaf of the left block, c the first leaf of the right block. The min in (R) therefore ranges over a set of pairs that *covers* every such order. It does **not** enumerate every internal flip configuration: for each candidate seam it substitutes the already-minimized $F_L(u, b)$ and $F_R(c, w)$. Whether that substitution preserves the global min is exactly the optimal-substructure question (Section 5). It is *not* granted by the existence of a nested min.

Orders with the right child on the left are **not** stored in M_v. They are represented only by the transpose fill of F_v. That representation is correct exactly when the relevant subtree optimum table is reversal-invariant, that is, when the true endpoint-conditioned optimum satisfies $C_v(x, y) = C_v(y, x)$ for every feasible endpoint pair. Symmetry $d = d^\top$ implies this property for every subtree, but it is stronger than necessary on a fixed instance.

2.7 Are all transitions valid?

A finite sum $F_L(u, b) + F_R(c, w) + d(b, c)$ corresponds to concatenating a T-consistent order of L with endpoints (u, b) and a T-consistent order of R with endpoints (c, w). The concatenation is T-consistent for v with $\phi(v) = 0$. No infeasible layout is generated by a *finite* transition.

The $+\infty$ entries of F are exactly the pairs of leaves that lie in the *same* child of the node (or the diagonal of a non-leaf). Those pairs cannot be the two endpoints of a binary node (Lemma 2.1), so excluding them does not drop a feasible layout.

Lemma 2.1 (cross-child endpoints)**.** *If $\lambda(v) \geq 2$, then in every T-consistent order of $L(v)$ the two endpoints lie in different children of v. Every pair $(u, w) \in L(\mathrm{lc}(v)) \times L(\mathrm{rc}(v))$* is *realized by some T-consistent order of $L(v)$.*

Proof. The two children occupy the two sides of the concatenation, so the left endpoint is in the left-placed child and the right endpoint in the right-placed child. Conversely, any leaf of a subtree can be made its left endpoint (place the unique child containing it on the left, recurse) and, independently, any leaf of the other child can be made the right endpoint. Independence holds because the two children's internal flips do not interact. □

2.8 Multiple representations of the same feasible solution

A single global order π induces, at each v, a unique pair of endpoints of the block $L(v)$ and a unique orientation of v's children. So a given π occupies **exactly one** stored cell of M_v *or* its transpose image in F_v, not both as independently optimized objects. Under $d = d^\top$ those two cells are forced equal, so double-counting is only a symmetry, not an alternate cost. Under asymmetric d the forced equality is a *false identification of two different layouts.*

This does not by itself cause overcounting of the reported min (the root min is taken over stored cells only), but it *does* cause the stored number to be the cost of a different orientation than the one reconstructed, whenever the transpose fill was used as an input to (R). That is an invalid transition under asymmetric d.

2.9 Can (R) exclude a globally optimal solution?

Two exclusion mechanisms exist.

1. **Missing orientation at a node, compensated by transpose.** Harmless iff $d = d^\top$; fatal otherwise (Section 3).
2. **Replacing a sub-arrangement by a cheaper one with the same endpoints.** Harmless *if* the state (v, u, w) is sufficient (Section 4). Harmful if the state forgets an endpoint (Section 4, 5).

No other pruning is performed. In particular the code does not implement the Bar-Joseph similarity-bound early exit.

2.10 Can (R) generate an infeasible solution?

Traceback concatenates child orders along witnesses of (R). By Section 2 every finite witness is a feasible concatenation. The reconstructed permutation is therefore T-consistent, **provided** the witnesses were computed from correct child tables. Under asymmetric d the child tables themselves contain transpose-fill values that do not equal any feasible child-path cost; traceback still emits a feasible permutation, but the *number stored in the table need not equal that permutation's true J* (Counterexample 3.4).

2.11 Can the reported value beat the true optimum?

Yes, but only through the invalid transpose identification. Under symmetric d, every stored number is a feasible-path cost (Theorem 3.1), hence cannot undershoot. Under asymmetric d, the stored min can be strictly smaller than $\min\{J(\pi) : \pi\ T\text{-consistent}\}$ and need not equal J of the reconstructed order (Counterexample 3.4, the $n = 4$ directed instance).

3 Global exactness

3.1 Statement

Theorem 3.1 (exactness under symmetry)**.** *Assume T is rooted and strictly binary, and $d = d^\top$ with entries in $\mathbb{R}$ (no* NaN*). Then* `olo_phylo` *returns a T-consistent order π with $J(\pi)$ equal to the global minimum of J over all T-consistent orders.*

Theorem 3.2 (no unconditional guarantee for asymmetric input)**.** *If arbitrary asymmetric d is allowed, the algorithm is not globally exact as an algorithmic guarantee. It can (i) return a suboptimal feasible order and, on larger instances, (ii) report a scalar that is not the J-value of the reconstructed order because transpose-filled child costs can be fictitious. The smallest counterexample to global optimality already has $n = 2$.*

More precisely, symmetry of d is **sufficient but not instance-wise necessary**. The weakest clean state-level condition is reversal invariance of every true subtree endpoint table C_v:

$$C_v(x, y) = C_v(y, x)$$

for every internal v and every feasible endpoint pair. Under this condition the transpose fill is valid and the proof of Theorem 3.1 goes through verbatim. Outside this condition the code may still be accidentally correct on a particular instance, but no general guarantee remains.

Theorem 3.3 (binary-domain restriction)**.** *The implementation requires every internal node to have* ***outdegree exactly 2*** *in the rooted* `phylo` *representation. If a node has any other number of children, it stops. This is a domain restriction of recurrence (R): the transition is written for the concatenation of exactly two child blocks. A multifurcating node with k children introduces $k!$ possible child orders (before accounting for descendants), so the feasible family and the transition must be generalized rather than silently resolved. Calling* `multi2di()` *first changes that feasible family by choosing a binary refinement.*

The rest of this section proves 3.1 and 3.2. Non-negativity of d is **not** used.

3.2 Invariant

Invariant $I(v)$**.** After `dp(v)` returns,

1. For every $u \in L(\mathrm{lc}(v))$, $w \in L(\mathrm{rc}(v))$, $M_v(u, w)$ equals the minimum internal path cost of a T-consistent arrangement of $L(v)$ that starts at u, ends at w, and places $\mathrm{lc}(v)$ on the left.
2. $F_v(x, y)$ equals the minimum internal path cost of a T-consistent arrangement of $L(v)$ that starts at x and ends at y, over *both* child orientations — **provided** $d = d^\top$. If x, y is not a cross-child pair, $F_v(x, y) = +\infty$.
3. The witnesses B_v, C_v realize the min in (R).

3.3 Proof of Theorem 3.1

The argument is induction on $\lambda(v)$, then a root-finalization step. Throughout, $d = d^\top$.

Base. $\lambda(v) = 1$: $I(v)$ holds ($F = 0$ on the unique pair).

Inductive step. Let L, R be the children of v, and assume $I(L)$ and $I(R)$. Fix $u \in L(L)$, $w \in L(R)$.

Soundness ($\geq$). Let σ be any T-consistent arrangement of $L(v)$ with $\mathrm{lc}(v)$ on the left, start u, end w. Let (b, c) be its seam. The left block is a T-consistent arrangement of $L(L)$ with endpoints (u, b); by $I(L)$ its internal cost is at least $F_L(u, b)$. Likewise the right block costs at least $F_R(c, w)$. The unique cross edge contributes $d(b, c)$. Hence $J(\sigma) \geq$ the right-hand side of (R), so $J(\sigma) \geq M_v(u, w)$.

Achievability ($\leq$). Let $(b^\star, c^\star)$ be a minimizer of (R). By $I(L)$ there is a T-consistent arrangement of $L(L)$ with endpoints $(u, b^\star)$ of cost $F_L(u, b^\star)$; similarly for R. Concatenating them is T-consistent for v, starts at u, ends at w, and has cost exactly the right-hand side of (R). Lemma 2.1 guarantees that every stored pair (u, w) has at least one finite seam (e.g. any valid endpoints of the two children), so $M_v(u, w) < +\infty$.

Thus (1) holds at v.

Transpose fill, using symmetry. Take a T-consistent arrangement σ of $L(v)$ with start x and end y. Its reverse $\mathrm{rev}(\sigma)$ is T-consistent, starts at y, ends at x, and has $J(\mathrm{rev}(\sigma)) = J(\sigma)$ because

$d = d^\top$. Therefore the min cost with endpoints (x, y) equals the min cost with endpoints (y, x). Combined with (1) at v and at the two possible orientations, this is (2).

Witnesses (3) are stored by construction.

Finalization at the root. The stored cells of M_ρ are exactly the layouts with $\mathrm{lc}(\rho)$ on the left. Every T-consistent global order either has that orientation or is the reverse of one that does. Under $d = d^\top$ the two classes have identical cost sets, so $\min M_\rho = \min J$. Traceback, by (3) and the inductive construction, emits an order realizing that cell.

Completeness, soundness, optimality, termination — mapped to the requested skeleton.

1. **Completeness.** Every feasible π induces, at each v, some endpoint pair and orientation. That pair is a stored cell or its reverse. The min in (R) ranges over all seams of that cell. Induction therefore represents every feasible π by a path of states.

2. **Soundness.** Every finite state/transition is a concatenation of T-consistent child orders (Section 2).

3. **Optimality of retention.** For a fixed (v, u, w), only the min cost is kept. The exchange argument of Section 4 shows that discarding worse same-endpoint arrangements cannot drop a global optimum.

4. **Termination.** The recursion follows the tree, which is finite and acyclic. The reported root cell is the global min by the finalization paragraph.

3.4 Proof of Theorem 3.2, with the smallest counterexample

Counterexample 3.4 (minimal, $n = 2$, asymmetric d)**.** *Let the tree be the single cherry $T = (1, 2)$, and let*

$$d(1, 2) = 5, \qquad d(2, 1) = 1.$$

There are exactly two feasible orders:

order	*true cost*
$1, 2$	5
$2, 1$	1

The implementation stores only the root cell $M_\rho(1, 2) = 5$, takes its minimum, and returns order $1, 2$ with cost 5. The global optimum is the reverse order $2, 1$ with cost 1. Thus $n = 2$ is already sufficient to refute exactness for arbitrary asymmetric matrices.

*This example isolates a **root-finalization** failure: the code searches one root orientation and relies on reversal to cover the other. No child transpose is even needed.*

Ghost-cost failure requires a larger tree. Once an internal child table is used through the transpose fill, the stored scalar can cease to be the cost of the path later reconstructed. For example, on $T = ((1, 2), (3, 4))$, choose asymmetric entries so that $d(2, 1) \neq d(1, 2)$ and/or $d(4, 3) \neq d(3, 4)$. The child table stores the forward value but the parent may consume it as though it were the reverse value. A parent transition can therefore add costs belonging to mutually inconsistent orientations. The traceback still emits a topology-feasible order, but the reported scalar need not equal that order's true directed path cost.

This distinction matters:

1. $n = 2$ proves **suboptimality** under asymmetric d.
2. $n \geq 4$ can exhibit the stronger **cost/order inconsistency** caused by a transpose-filled child state.

The defect is entirely the unproved reversal identification; storing both ordered endpoint orientations removes it while preserving the same DP logic.

3.5 What the intended pipeline actually satisfies

`make_trait_dist` calls `dist()`, which returns a symmetric matrix. For every input that follows the documented driver (`scale` + Euclidean / Minkowski `dist` on real traits without `NaN`), Theorem 3.1 applies and the algorithm is globally exact.

Exactness is **conditional**, not universal. Symmetry $d = d^\top$ is the simple, intended sufficient condition, not "d comes from traits". A caller who passes an asymmetric Kullback–Leibler matrix, a directed hit-and-miss score, or a raw non-symmetric matrix leaves the guaranteed-correct domain. Such an instance may happen to be solved correctly, but the algorithm no longer has a correctness guarantee (Counterexample 3.4).

4 Minimal sufficient state

4.1 The candidate and the true state index

After processing a subtree block, every objective term outside that block can interact with its internal arrangement only through the two leaves exposed at its ends. Thus the natural interface is the endpoint pair.

For a fixed tree, the node identifier is actually **derivable** from the two endpoints. In every feasible non-leaf state the endpoints lie in different children of their subtree root, hence

$$v = \mathrm{LCA}_T(u, w).$$

Therefore the mathematically minimal generic state *index* for the standard subtree-composition DP is:

- under symmetric d: the unordered endpoint pair $\{u, w\}$;
- under directed/asymmetric d: the ordered endpoint pair (u, w).

The DP value $F_v(u, w)$ is the **value stored at the state**, not an additional decision-state coordinate. The implementation organizes these pair states by their derived node v, which is convenient but logically redundant.

Equivalently, one may keep the notation (v, u, w) for readability, with the invariant $v = \mathrm{LCA}(u, w)$. The proof below is about sufficiency of the endpoint interface, not about a particular hash key or array layout.

4.2 Classification of each component

Component	Classification
Node v	**Derivable** from u, w
Endpoint identity u	**Necessary** in a generic compositional summary
Endpoint identity w	**Necessary**
Order of the pair	**Redundant under symmetric** d; **necessary in general**
DP value $F_v(u, w)$	**Payload, not a state coordinate**
Full internal order	**Redundant for value optimization** once endpoints and optimal value are known
Flip vector inside v	**Redundant** for future costs
Ancestor context	**Redundant** for this subtree summary
Same-child endpoint pairs	**Infeasible** and removable

This establishes minimality **within the natural compositional model that summarizes an already-solved subtree for arbitrary future parent contexts**. It does not imply that every exact algorithm must materialize all endpoint states simultaneously. Section 7.3 discusses an asymptotically optimal $O(n^2)$ algorithm using a different sweeping decomposition and only linear additional working memory.

4.3 Proof that (v, u, w) is sufficient

Theorem 4.1. *Let σ, σ' be two T-consistent arrangements of $L(v)$ with the same endpoints (u, w). For every T-consistent global order π whose restriction to $L(v)$ is σ, the order π' obtained by substituting σ' for that interval is T-consistent, and*

$$J(\pi') - J(\pi) \;=\; J(\sigma') - J(\sigma).$$

*In particular, if $J(\sigma') \le J(\sigma)$, then π' is at least as good as π. Therefore any two partial solutions with the same (v, u, w) have **identical optimal continuation behaviour**: the continuation sees only u, w, and the already-paid internal cost.*

Proof. Substituting σ' does not change which set occupies which interval (still $L(v)$), and σ' is T-consistent for v, so π' is T-consistent. The adjacent pairs of π that are not internal to the block are: the pair immediately left of u (if any) and the pair immediately right of w (if any). Both involve u or w and a leaf outside $L(v)$, hence are identical for π and π'. All other external pairs are unchanged. The difference in J is therefore exactly the difference of internal path costs. □

4.4 Proof that neither endpoint can be dropped

Theorem 4.2 (both endpoint identities are necessary in the generic exact state)**.** *Even for symmetric d, a state that retains only one endpoint of a subtree is not sufficient for exact continuation. Under asymmetric d, even an unordered pair $\{u, w\}$ is insufficient because orientation matters.*

Proof, dropping the right endpoint while retaining the left. Consider the subtree

$$S = ((1, 2), 3)$$

inside the larger tree $T = (S, 4)$. Use symmetric distances with

$$d(3, 1) = 0, \quad d(1, 2) = 0, \quad d(3, 2) = 1,$$

$$d(2, 4) = 100, \quad d(1, 4) = 0,$$

and choose all other unspecified distances nonnegative and large enough that they do not create a cheaper competing case for the particular parent state considered below.

Two feasible arrangements of S have the **same left endpoint** 3:

$$\sigma_A = (3, 1, 2), \qquad J(\sigma_A) = 0,$$

$$\sigma_B = (3, 2, 1), \qquad J(\sigma_B) = 1.$$

Their right endpoints are respectively 2 and 1. If a compressed state indexed only by $(S, \text{left} = 3)$ keeps the cheaper internal layout, it keeps σ_A and discards σ_B. But at the parent, with leaf 4 placed to the right,

$$J(\sigma_A \| 4) = 0 + d(2, 4) = 100,$$

whereas

$$J(\sigma_B \| 4) = 1 + d(1, 4) = 1.$$

Thus the internally worse partial solution becomes strictly better after continuation. The forgotten right endpoint changes the seam cost and is therefore necessary.

Dropping the left endpoint is symmetric: reverse the construction (or attach the external leaf on the left). Hence neither endpoint identity can be removed from a generic exact state. □

Unordered pair $\{u, w\}$. Under symmetric d, $F_v(u, w) = F_v(w, u)$, so an unordered pair plus a single cost is sufficient if the combination logic is allowed to choose which endpoint is exposed on each side. The implementation realizes the same reduction more concretely by storing one cross-child orientation and obtaining the reverse by `rev()`. Under asymmetric d, the ordered states (u, w) and (w, u) are not equivalent in general.

4.5 Are all cross-child pairs necessary?

Theorem 4.3 (no generic pair can be dropped)**.** *For a node v with children L, R, and for generic d, every pair $(u, w) \in L(L) \times L(R)$ is the unique minimizing interface of $L(v)$ for some parent context. Hence the $\lambda(L) \cdot \lambda(R)$ stored cells are necessary in the worst case over d.*

Proof sketch. Give u a unique external left neighbour x with $d(x, u) = 0$ and $d(x, \cdot) = M$ huge otherwise; give w a unique external right neighbour y similarly; set internal distances so that every arrangement of $L(v)$ has comparable internal cost. The unique optimum of the three-block instance $x|L(v)|y$ uses endpoints (u, w). A DP that had dropped that cell would be forced to use a costly external adjacency. □

This rules out "keep only $O(\lambda(v))$ endpoints per node" as a *uniform* exact reduction. Special trees (combs: Section 7.5) *do* have fewer feasible pairs, and a DP that iterates only finite F-entries automatically exploits that; the current nested loops do not.

5 Optimal substructure

Analyse the *problem*, not the code.

5.1 What holds

Theorem 5.1 (optimal substructure for the endpoint state)**.** *Let $\pi^\star$ be a globally optimal T-consistent order, v any node, and (u, w) the endpoints of the interval $\pi^\star$ occupies on $L(v)$. Then the restriction $\pi^\star|_{L(v)}$ is an optimal arrangement of $L(v)$ among those with endpoints (u, w).*

Proof. Immediate from Theorem 4.1: a cheaper same-endpoint restriction could be substituted, preserving feasibility and strictly decreasing J, contradicting global optimality of $\pi^\star$. □

This is the Bellman property that (R) relies on. It uses:

- additivity of J along the path,
- Lemma 1.1 (no edges skip over the block),
- the state containing *both* endpoints.

It does **not** use non-negativity, the metric axioms, or symmetry. Symmetry is used only to share $F(u, w)$ with $F(w, u)$.

Overlapping subproblems. At the endpoint-state level, yes: a child value $F_v(u, w)$ is reused by many parent seam candidates. At the *node-recursion* level, however, the rooted tree has unique parents, so `memo[[node]]` is not needed to prevent repeated recursive visits when starting once at the root; it merely caches the already-built table. The true state-dependency graph has vertices equal to endpoint-conditioned states and hyperarcs corresponding to seam combinations. It is acyclic because every dependency goes from a subtree to a strict descendant subtree.

Markov / sufficient-state structure. Theorem 4.1 is exactly that: the future is conditionally independent of the internal arrangement given (v, u, w) and the paid cost.

Greedy-choice property. False. See Sections 5.2–5.3. Consequently a "flip each node to minimize the present seam, given already-chosen child orders" algorithm is not exact, even though it inspects a DP-like tree.

5.2 What fails: endpoint-free "optimal subtrees"

The statement

> every optimal solution restricts to an *unconstrained* optimal solution of each subtree

is **false**. The restriction is optimal only *in the slice of fixed endpoints.*

Theorem 5.2. *The problem does **not** have optimal substructure with respect to the state "subtree v" (a single best order per node).*

5.3 Minimal counterexample (also the necessity witness for endpoints)

Counterexample 5.3. *Tree* $T = \big(1, ((2,3),4)\big)$. *Symmetric distances:*

$$d(2,3) = 1, \quad d(3,4) = 1, \qquad\qquad d(2,4) = 0.5,$$
$$d(1,2) = 0, \quad d(1,3) = d(1,4) = 100.$$

(All unspecified pairs are irrelevant because they never appear as T*-adjacencies.)*

Let $T_0 = ((2,3),4)$. *The* T_0*-consistent orders and internal costs:*

order of T_0	*internal* J	*endpoints*
$3,2,4$	$1 + 0.5 = 1.5$	$3,4$
$4,2,3$	$0.5 + 1 = 1.5$	$4,3$
$2,3,4$	$1 + 1 = 2$	$2,4$
$4,3,2$	$1 + 1 = 2$	$4,2$

An algorithm that keeps one best order of T_0 *keeps a cost-*1.5 *layout, with endpoints in* $\{3,4\}$. *Completing with leaf* 1*:*

- $1|3,2,4$ *costs* $100 + 1.5 = 101.5$*,*
- $1|4,2,3$ *costs* $100 + 1.5 = 101.5$*.*

The globally optimal order is $1,2,3,4$ *with* $J = 0 + 1 + 1 = 2$*, whose restriction to* T_0 *is the* locally suboptimal *layout* $2,3,4$*.*

The endpoint-aware DP computes $F_{T_0}(2,4) = 2$ *and* $F_{T_0}(3,4) = 1.5$*, then at the root*

$$M_\rho(1,4) \;=\; \min_c \big(F_{T_0}(c,4) + d(1,c)\big) \;=\; \min\big(2+0,\; 1.5+100\big) \;=\; 2,$$

and recovers $1,2,3,4$*. Independently, a "single best order per subtree" implementation on this instance returns cost* 101.5*.*

This counterexample is minimal: $n = 4$*, and on* $n = 3$ *every subtree is a cherry (internal cost has no alternative), so endpoint-free search happens to be exact for* $n \le 3$*.*

What additional state restores the property. Exactly the pair of endpoints, i.e. the state already used by (R) under Theorem 5.1.

6 Dominance relations

A dominance rule is safe only if every global optimum that used the discarded state has a matching (or better) global optimum that uses the retained state.

6.1 Rules that *are* safe

D1. Same-interface min (already in the DP). For fixed (v,u,w), only the smallest internal cost is kept. Safe by Theorem 4.1. This is scalar dominance on a singleton interface.

D2. Reversal identification, $d = d^\top$. The state (v,u,w) and (v,w,u) are equivalent. Safe by reversal invariance. This is a symmetry reduction of factor exactly 2 on non-leaf tables, and is already implemented.

D3. Infeasible same-child pairs. $F_v(x, y) = +\infty$ when x, y lie in the same child. Safe by Lemma 2.1. Already implemented (those cells are never written, and are skipped by the `is.finite` guard).

D4. Pointwise seam dominance (combination-time). Fix a node v with children L, R, and fix a left endpoint $u \in L(L)$. If $b, b' \in L(L)$ satisfy

$$F_L(u, b) \le F_L(u, b') \qquad \text{and} \qquad d(b, c) \le d(b', c) \quad \text{for all } c \in L(R),$$

then b dominates b' as a seam candidate for this u against this sibling R: every continuation that would have used b' can use b instead, and the objective does not increase. *Proof.* The combination formula (R) adds $F_L(u, \cdot) + d(\cdot, c) + F_R(c, w)$; the first two terms are never worse for b than for b', for every c, and F_R does not depend on b.

D4 does not change the worst-case complexity: for generic d the pointwise inequality on $d(b, \cdot)$ vs $d(b', \cdot)$ fails. It is a correct optional filter.

D5. Bound-based elimination inside the inner min (Bar-Joseph pruning, dual). For a *maximization* of similarities, Bar-Joseph et al. sort candidate seams and stop when a remaining candidate cannot beat the current best given an upper bound on the best unused similarity. The min-cost dual is: given a current best B for $M_v(u, w)$ and a lower bound $\ell(b)$ on $\min_c\big(F_R(c, w) + d(b, c)\big)$, skip b when $F_L(u, b) + \ell(b) \ge B$. Safe. The supplied code does **not** implement it. Worst-case complexity is unchanged (adversarial d never prunes); typical complexity can drop, as in the original Bioinformatics experiments.

6.2 Rules that *look* safe and are not

U1. Scalar dominance across different interfaces. "If $F_v(u, w) \le F_v(u', w')$ then drop (u', w')." Unsafe. Different endpoints have different external options. Counterexample 5.3: $F_{T_0}(3, 4) = 1.5 < 2 = F_{T_0}(2, 4)$, but dropping $(2, 4)$ loses the unique global optimum.

U2. Same-end, cheaper-start (or same-start, cheaper-end) without a pointwise condition on d. "If $F_v(u, w) \le F_v(u', w)$ then drop (u', w)." Unsafe. The leftward continuation cares about u vs u'. In Counterexample 5.3, $F_{T_0}(3, 4) < F_{T_0}(2, 4)$ with a common right endpoint 4; dropping $(2, 4)$ again loses $1, 2, 3, 4$.

U3. Pareto dominance on a scalarization that mixes cost and a single "centrality" of the endpoint. Any scalar that collapses the vector $\big(d(w, c)\big)_{c \notin L(v)}$ can be fooled by putting the only cheap external neighbour on a coordinate that the scalar underweighted. The safe version is D4's *pointwise* (Pareto) comparison, which is high-dimensional and rarely strict.

U4. Greedy seam: at each node keep only the orientation whose *current* child-orders give the cheaper seam. This is the endpoint-free algorithm of Section 5.2. Unsafe by Counterexample 5.3.

U5. Reversal identification when $d \ne d^\top$. Unsafe by Theorem 3.2. Dominating (w, u) by (u, w) discards a genuinely different directed layout.

6.3 Resource / bound dominance in a branch-and-bound on flips

Branch-and-bound is exact whenever its lower bound is **admissible**: it must never exceed the minimum possible completion cost of the current partial choice.

For nonnegative d, zero is an admissible but weak remainder bound. An MST relaxation is also admissible when constructed on the still-required vertices (and any required interface terminals), because every feasible Hamiltonian path completion is a spanning tree after forgetting order and laminar constraints; the minimum spanning tree can only be cheaper. The OLO constraints make

the feasible completion set smaller, which strengthens the validity of the relaxation rather than invalidating it.

What an MST bound usually fails to capture is precisely the expensive part of this problem: required endpoint identities and laminar block compatibility. It can therefore be weak. Stronger admissible bounds can incorporate interface constraints, degree relaxations, assignment relaxations, or exact DP values for unresolved subtrees. None changes the worst-case fact that branch-and-bound may explore exponentially many flip assignments.

Thus branch-and-bound can be useful on favorable instances, but it is not the best worst-case formulation: bounded-degree OLO admits a quadratic-time exact algorithm (Section 7.3).

7 Stronger recurrences and alternative formulations

The existing recurrence is correct (under $d = d^\top$) but not computationally tight. The improvements below are mathematically justified; novelty for its own sake is avoided.

7.1 Factored min-plus form — same exactness, $O(n^3)$

Distribute the min in (R):

$$M_v(u,w) \;=\; \min_{b\in L(L)}\Big(F_L(u,b) \;+\; \min_{c\in L(R)}\big(F_R(c,w)+d(b,c)\big)\Big). \tag{2}$$

Lemma 7.1. *Over $\mathbb{R}\cup\{+\infty\}$, (R) and (2) are identical.*

Proof. $\min_{b,c}\big(A(b)+B(c)+D(b,c)\big) = \min_b\big(A(b)+\min_c(B(c)+D(b,c))\big)$ by associativity of min and independence of $A(b)$ from c. □

The inner map $(w,b)\mapsto \min_c\big(F_R(c,w)+d(b,c)\big)$ does **not** depend on u and is computed once, in time $\Theta\big(\lambda(L)\,\lambda(R)^2\big)$. The outer min is then $\Theta\big(\lambda(L)^2\,\lambda(R)\big)$. Per node this is $\Theta\big(\lambda(L)\,\lambda(R)\,\lambda(v)\big)$ rather than $\Theta\big(\lambda(L)^2\,\lambda(R)^2\big)$.

This is the standard improvement that makes the cba package's `order.optimal` an $O(n^3)$ algorithm; the supplied code implements the unfactored $O(n^4)$ quadruple loop.

(2) is **not** a different state. It is the same state with an equivalent transition, and Theorem 3.1 transfers verbatim.

In matrix language: if $A = F_L$, $B = F_R$, $D = d|_{L(L)\times L(R)}$, then $M_v = A\otimes D\otimes B$ in the min-plus semiring. (2) is parenthesization $(A\otimes D)\otimes B$ vs $A\otimes(D\otimes B)$, both $O(n^3)$ at a balanced root.

7.2 Directed (asymmetric) exact DP

Drop the transpose fill. Store both orientations as independent cells:

$$M_v^{\rightarrow}(u,w), \qquad M_v^{\leftarrow}(w,u),$$

and use the true $F_L(u,b) = M_L^{\text{oriented}}(u,b)$ without identifying reversals. Transitions stay (R) / (2), with $d(b,c)$ *not* replaced by $d(c,b)$. Exact for arbitrary real d, at a constant-factor increase in stored orientation states and the same asymptotic time. This claim follows from the same endpoint-conditioned induction as Theorem 3.1, now without identifying reverse states.

This is the minimal change that restores Theorem 3.1 without the hypothesis $d = d^\top$.

7.3 Stronger general result: an asymptotically optimal $O(n^2)$ exact algorithm

The cubic endpoint-table DP is **not** asymptotically optimal for arbitrary binary trees.

Brandes (2007), while reviewing prior work, states that for a tree of maximum node degree d, Deineko and Tiskin gave an exact algorithm with

$$O(4^d n^2) \text{ time}, \qquad O(4^d n) \text{ additional space}.$$

For binary trees, d is a fixed constant, hence this becomes

$$O(n^2) \text{ time}, \qquad O(n) \text{ additional space}.$$

This matches the natural $\Omega(n^2)$ lower bound in the explicit-distance-matrix model.

Theorem 7.2 (quadratic input-query lower bound)**.** *In a model where the entries of an arbitrary symmetric dissimilarity matrix are supplied explicitly or by random-access queries, every exact algorithm has worst-case $\Omega(n^2)$ input/query complexity.*

Proof. Run the algorithm on the all-zero symmetric matrix. Suppose some off-diagonal pair $\{i, j\}$ is never inspected. Let $v = \mathrm{LCA}(i, j)$. By orienting the child of v containing i so that i is its rightmost leaf, orienting the other child so that j is its leftmost leaf, and choosing arbitrary feasible orientations elsewhere, there exists a tree-consistent order in which i, j are adjacent. Now form a second input identical on all queried entries but set $d(i, j) = d(j, i) = -1$. The optimum of the all-zero input is 0, while the modified input has a feasible order of cost at most -1. The algorithm sees the same answers to every query on both inputs and therefore cannot be correct on both. Hence in the worst case every unordered pair must be inspected, giving $\Omega(\binom{n}{2}) = \Omega(n^2)$. □

The quadratic algorithm comes from the PQ-tree / minimum-weight tree- shortcutting viewpoint rather than from storing every endpoint table and performing a min-plus product at each internal node. Its core uses **sweeping paths** and specialized layered acyclic graphs. For bounded degree, the number of child-subset configurations is constant; a global counting argument bounds the relevant triples/quadruples by $O(n^2)$, and shortest paths in the layered DAGs are computed in time proportional to their arcs. This is a genuine change of decomposition, not merely a faster implementation of (2).

The quadratic result is cited here through Brandes specifically as an exact algorithmic result for tree-compatible **leaf ordering**. Therefore no auxiliary reduction from this linear-path OLO objective to a cyclic TSP objective is needed for the complexity conclusion. This avoids importing extra representability assumptions about a dummy terminal into the fixed-tree feasible family.

Consequence. The theoretically best asymptotic target on binary trees is quadratic, not cubic. The endpoint-table recurrence remains attractive because it is considerably simpler to prove and implement, but it is not state of the art in worst-case complexity.

7.4 Brandes (2007): practical $O(n^2 \log n)$ for complete / near-complete trees

For complete binary trees, and more generally trees of height $\log_2 n + O(1)$ after controlled completion, Brandes gives a particularly simple algorithm using canonical bit labels and position-based predecessor sets. It runs in $O(n^2 \log n)$ time and $O(n)$ additional space.

Its key positional lemma fails for incomplete trees: fixing a leaf at a position no longer uniquely determines the partition of the remaining leaves. Padding a height-h tree to completeness may create 2^h leaves; for a comb this is exponential. Thus Brandes is a practical specialization, not the general quadratic method of Section 7.3.

7.5 Comb (caterpillar) specialization: $O(n^2)$

If $T_k = (T_{k-1}, k)$, leaf k is an endpoint of every arrangement of T_k. The only nontrivial state is which older leaf forms the opposite endpoint. There are $O(k)$ such states at layer k, and the transition can be restricted to the finite support of the preceding comb table, yielding $O(n^2)$ total time.

The supplied code still spends $\Theta(n^3)$ on a comb because it scans many infinite entries in dense full matrices. Iterating only over finite support recovers the quadratic behavior on this shape.

7.6 Alternative formulations: what applies and what does not

Shortest path / DAG. A shortest-path interpretation *does* help, but not as "run Dijkstra on the leaves." The useful graph is a layered **state graph** of sweeping configurations, as in the quadratic bounded-degree algorithm. Because it is acyclic, topological shortest-path DP is preferable to Dijkstra and also works with negative edge weights.

The endpoint recurrence (R) can likewise be viewed as a shortest hyperpath in an acyclic state hypergraph. That viewpoint alone does not reduce complexity; the improvement comes from choosing a state graph with only $O(n^2)$ relevant transitions for bounded degree.

PQ-tree formulation. This is directly relevant. A rooted binary tree that allows only child swaps is a restricted PQ-tree. Classical PQ-tree TSP machinery gives a polynomial exact formulation, and the later tree-shortcutting specialization is what reaches quadratic time for bounded degree.

Min-cost flow. No comparably direct compact network-flow formulation is needed to obtain the optimal asymptotic bound. Encoding "visit each leaf once" and laminar contiguity through a generic flow model tends to reintroduce TSP-like degree/order structure. Since $\Omega(n^2)$ time is already forced by an explicit arbitrary cost matrix, a flow formulation could at best match, not beat, the input-reading bound.

Held–Karp / meet-in-the-middle. These are exact but waste the laminar tree structure. Held–Karp ranges over arbitrary subsets and is exponential; meet-in-the-middle over the $n-1$ flip bits is also exponential. Both are inferior to the tree-specific polynomial algorithms.

Knuth/Yao, divide-and-conquer optimization, SMAWK/Monge. These require additional monotonicity or quadrangle/Monge structure on a *known* indexing. The leaf order here is itself the decision variable, and a generic Euclidean trait-distance matrix is not Monge in the arbitrary tip labelling. They may apply to special structured distance families, but not to the stated general problem without extra assumptions.

Circular OLO. Adding a wrap-around term gives a different objective, but can be handled by the endpoint machinery or the tour/PQ-tree formulation. It is not an acceleration of the linear objective.

7.7 How close is each formulation to optimal?

- supplied literal R recurrence: worst-case $\Theta(n^4)$;
- factored endpoint DP (2): $\Theta(n^3)$ worst-case;
- Brandes on complete/near-complete trees: $O(n^2 \log n)$, linear additional space;
- bounded-degree tree-shortcutting/PQ-tree algorithm: $O(n^2)$ time and $O(n)$ additional working space for binary trees;

- explicit arbitrary $n \times n$ dissimilarity input: $\Omega(n^2)$ time in the standard RAM/query model.

Hence the general bounded-degree quadratic algorithm is asymptotically optimal with respect to explicit-input size. There is no cubic barrier.

8 Complexity

8.1 The implementation as written (R)

Let $a_v = \lambda(L)$ and $b_v = \lambda(R)$ for the two children of an internal node v. Let q_L denote the number of **finite ordered endpoint pairs** in the full child table F_L. For a leaf child, $q_L = 1$. If L is internal with grandchildren of sizes p and q, then $q_L = 2pq$, because exactly the cross-grandchild ordered pairs are feasible.

The literal R implementation is not a completely dense four-loop evaluation, because it tests `is.finite(Lcost)` before constructing the length-b_v vector over c. Its work at node v separates into:

1. $\Theta(a_v^2 b_v)$ scalar `(u,w,b)` checks;
2. $\Theta(a_v b_v^2)$ extraction/scanning of the length-b_v `Rcol` vectors across all (u, w) pairs (up to constant and R-copy semantics);
3. $\Theta(q_L b_v^2)$ arithmetic and `which.min` work for the finite (u, b) pairs, because each such pair is revisited for every w and scans all $c \in L(R)$.

Thus a support-aware arithmetic model is

$$T_v^{\text{literal}} = \Theta\big(a_v^2 b_v + a_v b_v^2 + q_L b_v^2\big),$$

up to implementation-level copying constants. A completely dense evaluation of recurrence (R), ignoring the `is.finite` shortcut, would instead cost $\Theta(a_v^2 b_v^2)$.

The distinction does **not** change the worst case. At a balanced node whose left child is itself balanced, $q_L = \Theta(a_v^2)$ and $(a_v, b_v) = \Theta(s_v)$, so that node costs $\Theta(s_v^4)$. Hence a perfectly balanced tree satisfies

$$T(n) = 2T(n/2) + \Theta(n^4) = \Theta(n^4).$$

Therefore the supplied implementation has **worst-case $\Theta(n^4)$ time**.

For a comb, the finite support of the large child is only linear but the `(u,w,b)` checks are still quadratic at layer s, giving $\Theta(s^2)$ per layer and $\Theta(n^3)$ total. Mixed tree shapes can be strictly cheaper than the dense formula $\sum_v a_v^2 b_v^2$, so that formula should be read as a dense-recurrence bound, **not** as an exact count for every execution of this R code.

Space. The number of stored rectangular DP cells is

$$\sum_v a_v b_v = \binom{n}{2},$$

because each unordered pair of leaves is separated for the first time at its unique LCA. Hence all DP tables and traceback pointers use $\Theta(n^2)$ words. The temporary full child matrices can also reach $\Theta(n^2)$, so total working memory is $\Theta(n^2)$.

8.2 Factored transition (2)

Per node $\Theta(a_v b_v s_v)$. Using $s_v \le n$ and the identity above,

$$T_{(\mathrm{R3})}(T) \;=\; \Theta\Big(\sum_v a_v b_v s_v\Big) \;\le\; n \sum_v a_v b_v \;=\; O(n^3).$$

The bound is tight: the root of a balanced tree already contributes $\Theta(n^3)$. The same $\Theta(n^3)$ holds for a comb ($\sum_s s \cdot 1 \cdot s = \Theta(n^3)$) unless finite-entry iteration or the comb specialization is used.

Space is unchanged: $\Theta(n^2)$.

8.3 Special cases, pruning, typical

Setting	Time	Notes
(R), worst case	$\Theta(n^4)$	balanced tree, generic
(R), comb	$\Theta(n^3)$	R loops; $\Theta(n^2)$ if only
(R3), any binary tree	$\Theta(n^3)$	tight for this recurrenc
(R3), comb, finite-entry loops	$\Theta(n^2)$	equivalent to Section 7
Brandes, complete / height $\log n + O(1)$	$O(n^2 \log n)$	$O(n)$ additional space
Deineko–Tiskin bounded-degree tree shortcutting	$O(n^2)$ for binary trees	asymptotically optima
D4 / D5 pruning	still $\Theta(n^3)$ worst case	typical-case only
Endpoint-free greedy	$\Theta(n)$ after child orders, but **inexact**	not a competitor
Brute force on ϕ	$\Theta(n\, 2^{n-1})$	used as an oracle, not

Where the objective adds a factor. The $\lambda(L)\lambda(R)$ interfaces are forced by Theorem 4.3. The remaining $\times\lambda(v)$ in (2) is the cost of minimizing over the seam against an *arbitrary* d. If d were e.g. rank-1 or a 1-D Euclidean matrix in a *known* order, that factor could drop; under the problem as stated it is inherent to the transition.

8.4 The $n = 500$ driver

The comments advertise a 500-tip example. Under (R) a balanced 500-leaf tree is $\sim 500^4/14 \approx 4 \times 10^9$ inner operations, nested in R interpreter loops — not a realistic run. Under (2) it is $\sim 500^3/4 \approx 3 \times 10^7$ at the root plus a geometric series, which is routine in C and uncomfortable in pure R. A quadratic bounded-degree implementation is theoretically better still, but much more complex to engineer. The exactness theorem does not depend on n; tractability does.

9 Exhaustive search as a validation tool

9.1 What brute force can establish

Enumeration of all 2^{n-1} flip vectors, for a *fixed* (T, d), computes the true $\min J$ on that instance. Agreement of a DP with that value on a collection of instances is **evidence**, not a proof of exactness for all (T, d).

It *can* establish:

- existence of counterexamples (one failing instance is a proof of inexactness — Theorems 3.2 and 5.2 were obtained this way);
- that two recurrences (R) and (2) agree on the tested set;
- that a proposed dominance rule is unsafe (a dominated state used by the unique optimum);
- that an implementation bug exists on small n.

9.2 What brute force cannot establish

- Exactness for all n. A bug that first appears at $n = 11$ (or on a tree shape not sampled, or on a measure-zero algebraic variety of d) will be missed. The authors' report of agreement for $n = 4..10$ is consistent with Theorem 3.1 but is not that theorem.
- Exactness for a reduced state. Agreement after dropping a cell on random d may mean only that the dropped cell was not needed *on those* d.
- Asymptotic complexity.

9.3 What *would* constitute a proof of equivalence between (R) and brute force

A proof that, for every (T, d) in the stated class,

$$\min M_\rho(T, d) \;=\; \min_{\phi \in \{0,1\}^{n-1}} J(\pi_\phi; d),$$

together with a proof that traceback emits a π_ϕ realizing the left side. That is Theorem 3.1. It is an induction on T, not a finite check.

A finite check *plus* a proof that "if it works for all trees with $n \le n_0$ then it works for all n" would require a reduction of large instances to small ones. No such reduction exists for this DP: the inductive step *is* the proof, and it does not factor through a bounded n.

9.4 A systematic counterexample methodology

The search that produced Sections 3 and 5.3, and that failed to break Theorem 3.1, is:

1. **Independent oracle:** enumerate π_ϕ, compute J in floating point, take the min. Do not reuse DP code in the oracle.
2. **Instance families:** (a) all binary tree shapes for $n \le 6$; (b) random recursive binary trees; (c) combs; (d) perfectly balanced; (e) the structured four-leaf tree of Counterexample 5.3.
3. **Matrices:** Euclidean traits of dimension $1, 2, 5, 10$; generic symmetric; generic asymmetric; all-zero; all-off-diagonal-equal; symmetric with negative entries; quantized (ties).
4. **Predators, not just agreement tests:**
   - endpoint-free greedy (must fail if endpoints are necessary);
   - (R) with transpose fill on asymmetric d (must fail if symmetry is necessary);
   - (2) vs (R) (must agree if the identity is implemented);
   - directed two-orientation DP vs brute (must agree if the directed DP is right).

5. **Tie and** NaN **probes.** Quantized d; constant trait columns (the `scale` hazard).

6. **Cost-vs-order consistency check:** the reported scalar must equal J of the reconstructed permutation, independently computed. This is what caught the "ghost cost" in Counterexample 3.4's $n = 4$ case.

A falsification of Theorem 3.1 would be any instance in class (rooted binary T, symmetric finite d) on which step 6 fails or the DP min differs from the oracle. None was found; that is evidence, and the proof is Section 3.

10 Counterexample search

Adversarial targets and outcomes.

10.1 Incorrect state compression

Endpoint-free compression is inexact: Counterexample 5.3, $n = 4$, gap 101.5 vs 2. The counterexample is already a theorem-level falsification; no failure frequency is needed for the conclusion.

Dropping one endpoint while keeping the other is inexact by Theorem 4.2; the same structured instance is a witness.

10.2 Missing history

Any history beyond (v, u, w) is redundant (Theorem 4.1). Search for a counterexample to *sufficiency* of that state: none exists under the exchange argument, which does not depend on n.

10.3 Invalid optimal-substructure assumption

The unconstrained-subtree version fails (Counterexample 5.3). The endpoint-sliced version holds (Theorem 5.1). The implementation uses the latter.

10.4 Invalid dominance

U1–U5 of Section 6 are each refuted by Counterexample 5.3 or 3.4.

10.5 Incorrect transition ordering

At the node level, any post-order is a valid evaluation order because every parent depends only on its descendants. At the finer endpoint-state level the dependency structure is an acyclic hypergraph. The recursive implementation respects this partial order automatically, so there is no transition that uses a child table before that child has been solved.

10.6 Boundary cases, ties, degeneracy

Probe	Result under (R) with symmetric d
$n = 2$ (single cherry)	exact: the only cost is $d(1, 2)$
$n = 3$	exact under symmetry; **inexact** without (Counterexample 3.4)
all-zero d	exact, cost 0, every order optimal
equal off-diagonal $d \equiv 1$	exact, cost $n - 1$
negative symmetric d	exact (non-negativity unused)
quantized d (many ties)	exact cost; arbitrary choice among optima
comb and balanced, $n \leq 8$	exact
random Euclidean/symmetric small instances	empirical agreement with brute force; evidence only
constant trait $\Rightarrow$ `NaN` after `scale`	**not exact: IEEE `NaN` poisons the min** (implementation hazard, outside Theorem 3.1's finite-d hypothesis)

10.7 Asymmetric / directed inputs

Random directed tests are useful only as falsification aids. The theorem-level smallest witness is Counterexample 3.4 with $n = 2$; a four-leaf balanced tree can additionally expose cost/order inconsistency through transpose-filled child states.

10.8 Locally inferior partial solutions that are globally superior

This is the *content* of Counterexample 5.3, and is exactly why the state must retain currently-suboptimal interfaces. The implementation does retain them (all cross-child pairs). The endpoint-free algorithm does not.

10.9 Multiple histories, same nominal state, different futures

Under the state (v, u, w), Theorem 4.1 says this cannot happen. Under a compressed state it does happen, and 5.3 is the picture.

10.10 What was *not* found

No counterexample to Theorem 3.1 on the class it claims. That is **not** a proof; the proof is Section 3. The search is reported here because the mandate was to try to break the algorithm *before* proving it, and the breakage that exists (asymmetric d, endpoint-free compression, `NaN`) was found by that search, not by accepting the recurrence.

11 Empirical evidence versus proof

Claim	Status
(R) is exact for rooted binary T and symmetric finite d	**Proven** (Theorem 3.1)
(R3) is equivalent to (R)	**Proven** (Lemma 7.1)
(v, u, w) is sufficient	**Proven** (Theorem 4.1)
neither endpoint is droppable	**Proven** (Theorem 4.2, Counterexample 5.3)
unconstrained-subtree optimal substructure fails	**Proven** (Theorem 5.2, Counterexample 5.3)
(R) is inexact for asymmetric d	**Proven** (Theorem 3.2, Counterexample 3.4)
D1–D5 are safe dominance rules	**Proven** (Section 6)
U1–U5 are unsafe	**Proven** by counterexample (Section 6)
$\sum_v a_v b_v = \binom{n}{2}$	**Proven** (LCA counting)
$T_{(\mathrm{R})} = \Theta(n^4)$ worst case; $T_{(\mathrm{R3})} = \Theta(n^3)$	**Proven** (Section 8)
Brandes' position-based algorithm does not extend directly to arbitrary-height trees	**Proven in Brandes' analysis**
a quadratic exact algorithm exists for bounded-degree trees	**Established in the literature** (Deineko–Tiskin, cited by Brandes)
independent small-instance brute force agrees with (R) on tested symmetric cases	**Empirical only** — supports, does not replace, Theorem 3.1
the code comment's claimed $n = 4..10$ brute-force agreement	**Empirical claim in the supplied material only**
D4/D5 help on typical trait data	**Conjectured / historically reported** (Bar-Joseph 2001 tables); not re-measured here
`rotateConstr` realizes the DP order on every ape tree	**Not proven here**; outside the DP. A mismatch would be an ape bug or a contract bug, not a recurrence bug
Euclidean trait d has extra structure that a specialized OLO could exploit	**Conjectured**; no algorithm given

No empirical row is promoted to a theorem in Section 12.

12 Final theoretical verdict

12.1 Exactness

For the intended domain — a rooted strictly binary tree and a finite symmetric matrix d — the algorithm is **globally exact** (Theorem 3.1). Symmetry is a clean sufficient condition because it makes every subtree optimum table reversal-invariant.

This is **not an instance-wise "if and only if" characterization**. Some asymmetric matrices can accidentally satisfy the needed reversal invariance. What is true is that arbitrary asymmetric input is outside the guaranteed correct domain: the smallest counterexample has $n = 2$ (Theorem 3.2), and larger instances can additionally create reported costs inconsistent with the reconstructed directed order.

Not defined for polytomies or unrooted trees; the implementation correctly rejects the former and silently assumes the latter are rooted.

Non-negativity, the triangle inequality, and the trait-Euclidean origin of d are **not** required for exactness.

12.2 Required state

For the standard **subtree-composition** DP, the minimal generic interface is the two endpoint identities. Under symmetric d, the state index can be the unordered pair $\{u, w\}$; under asymmetric d, it must be the ordered pair (u, w). The subtree node is derivable as $v = \mathrm{LCA}(u, w)$, and $F_v(u, w)$ is the value stored at the state rather than an additional state coordinate.

Both endpoint identities are necessary for arbitrary continuation contexts (Theorem 4.2); the full internal order and flip history are not. Theorem 4.3 shows that no generic endpoint cell can be pruned uniformly from this compositional DP, although a different exact algorithm need not materialize all such cells simultaneously.

12.3 Optimal substructure

Holds for the endpoint-conditioned state (Theorem 5.1) and fails for the endpoint-free state (Theorem 5.2). There is no greedy-choice property. Child endpoint values are reused across many parent transitions; the state-dependency structure is an acyclic hypergraph induced by the rooted tree, not merely the node tree itself.

12.4 Dominance

Safe to eliminate: duplicate reverse cells when $d = d^\top$ (D2); infeasible same-child pairs (D3); same-interface worse internals (D1); pointwise-seam-dominated candidates (D4); bound-pruned candidates in the inner min (D5).

Unsafe to eliminate: any cell solely because it has worse internal cost than a cell with a *different* interface (U1, U2, Counterexample 5.3).

12.5 Complexity

Algorithm	Time	Additional/working space	Domain
supplied literal R code	$\Theta(n^4)$ worst case	$\Theta(n^2)$	general binary,
factored endpoint DP (R3)	$\Theta(n^3)$ worst case	$\Theta(n^2)$	same
support-aware comb DP	$\Theta(n^2)$	shape-dependent	combs
Brandes 2007	$O(n^2 \log n)$	$O(n)$	complete / heig
Deineko–Tiskin bounded-degree algorithm	$O(n^2)$ for binary trees	$O(n)$	general bounde
brute force	$\Theta(n2^{n-1})$	$O(n)$ besides input	validation oracl

The tightest justified asymptotic bound for the **general binary-tree problem** with explicit pairwise costs is $\Theta(n^2)$ time: the literature gives an $O(n^2)$ exact bounded-degree algorithm, while reading/querying an arbitrary dense dissimilarity matrix requires $\Omega(n^2)$ in the standard model. The algorithm's extra working memory can be linear, although the supplied dense matrix d itself already occupies $\Theta(n^2)$ storage.

12.6 Potential improvements (theoretically justified, exactness-preserving)

1. **Immediate implementation change: replace (R) by (R3).** Same proof and state semantics, worst case $O(n^4) \to O(n^3)$; this is the simplest high-impact improvement to the supplied R code.

2. **Exploit finite support.** Do not materialize or scan `Inf` regions of child endpoint matrices. This gives strong shape-dependent savings and quadratic behavior on comb-like trees.

3. **For asymptotic optimality, change decomposition.** Implement the bounded-degree PQ-tree / tree-shortcutting algorithm: $O(n^2)$ time and $O(n)$ additional space for binary trees. It is mathematically stronger but much more complex than (R3).

4. **Use Brandes for complete or near-complete trees.** It is simpler than the general quadratic method and uses linear additional memory.

5. **Use safe dominance/bounds only as secondary accelerators.** D4/D5 and admissible branch-and-bound bounds may help typical instances but do not improve the endpoint DP's worst-case asymptotic guarantee.

6. **If directed costs are needed, store both endpoint orientations** rather than using the transpose shortcut.

7. **Guard `scale()` against zero-variance traits** so the distance matrix stays finite.

Do **not** drop an endpoint, keep only one order per subtree, or identify reversals for directed costs. Generic Dijkstra-on-leaves, Held–Karp, meet-in-the-middle, and standard Monge/Knuth optimizations do not exploit the right structure.

12.7 Remaining gaps

- No cubic lower bound applies to the problem: Section 7.3 already records an exact $O(n^2)$ bounded-degree algorithm. What remains open *within the simple endpoint-table/min-plus decomposition used by (R3)* is how far one can reduce its work without changing decomposition

or exploiting additional structure in d. The document therefore does **not** claim that $\Theta(n^3)$ is an intrinsic barrier for OLO.

- `ape::rotateConstr` is not part of the proof. Exactness is claimed for `olo_phylo`'s `(order, cost)`, not for the plotted embedding.
- Numerical stability of the `which(table == min_cost)` tie-break under floating-point noise is not analysed; it does not affect the *value* of the min, only which optimum is selected.
- Euclidean-trait structure (low-dimensional embeddings, identity-minus-correlation matrices, etc.) is unused. Whether it yields a faster *exact* algorithm is open.

12.8 Confidence

Theorem-level: exactness of (R) and (R3) on rooted binary trees with symmetric finite d; lack of a correctness guarantee for arbitrary asymmetric d; sufficiency and necessity of the two-endpoint state; failure of endpoint-free optimal substructure; safe/unsafe dominance classification D1–D5 / U1–U5; $\Theta(n^4)$ vs $\Theta(n^3)$ arithmetic complexity.

Empirical: supplementary small-instance brute-force checks are consistent with Theorem 3.1 on symmetric inputs and readily produce failures on directed inputs. These checks are validation aids only; none of the theorem-level claims relies on their success.

Literature-level conclusion: the general binary-tree problem admits an asymptotically optimal quadratic-time exact algorithm for explicit pairwise cost input. The cubic endpoint DP is simpler, but not asymptotically optimal.

References and external cross-checks

1. Z. Bar-Joseph, D. K. Gifford, and T. S. Jaakkola, "Fast optimal leaf ordering for hierarchical clustering," *Bioinformatics* 17(Suppl. 1), S22–S29, 2001. DOI: 10.1093/bioinformatics/17.suppl_1.S22. The paper presents the original practical OLO framework and an $O(n^4)$ basic algorithm with further improvements.

2. R. E. Burkard, V. G. Deineko, and G. J. Woeginger, "The Travelling Salesman and the PQ-Tree," *Mathematics of Operations Research* 23(3), 613–623, 1998; erratum 24(1), 262–272, 1999. It gives an $O(2^d n^3)$ exact algorithm for bounded-degree PQ-tree-restricted TSP.

3. V. G. Deineko and A. Tiskin, "Fast minimum-weight double-tree shortcutting for Metric TSP," WEA 2007 / later JEA version. The bounded-degree sweeping-path algorithm runs in $O(4^d n^2)$ time and $O(4^d n)$ working space; for binary trees this is quadratic time and linear additional space.

4. U. Brandes, "Optimal leaf ordering of complete binary trees," *Journal of Discrete Algorithms* 5(3), 546–552, 2007. DOI: 10.1016/j.jda.2006.09.003. Brandes recapitulates the binary-tree endpoint DP as $\Theta(n^3)$, cites the quadratic Deineko–Tiskin result, and gives a simple $O(n^2 \log n)$-time, $O(n)$-space method for complete / near-complete trees.

5. `cba::order.optimal` documentation states $O(n^3)$ time for its binary-tree OLO implementation.

These references support the historical and asymptotic context. In particular, Brandes (2007) explicitly states that tree-compatible ordering with bounded node degree is polynomial and cites the Deineko–Tiskin $O(4^d n^2)$-time, $O(4^d n)$-space exact result; for fixed-degree binary trees this is quadratic time. The proof of exactness of the supplied recurrence under symmetric finite d, the endpoint-sufficiency exchange argument, and the counterexamples in this document are self-contained.